

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_e2a9aaa7-6b2\agent-aca1550be93c2dafe.jsonl`  
- SHA-256: `fb4a07efda32e18bd2971b89e2302b6aaa163a69038d2f10dae900c75d3abf1d`  
- Source modified: `2026-05-30T19:18:40+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_e2a9aaa7-6b2`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-30T19:17:03.562Z)

Adversarial Claim Verifier (voter 2/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Find promising AND reliable open-source tools/libraries for parsing Indian bank statement PDFs ? especially HDFC Bank savings/transaction statements ? into structured transaction rows (date, narration, ref, value date, withdrawal, deposit, closing balance), for reuse in a LOCAL-FIRST Python personal-finance tool ("muneem").

HARD CONSTRAINTS (a tool that violates these is unusable for us):

1. Must run 100% LOCALLY. Bank statements are highly sensitive ? NO cloud APIs, NO paid SaaS (e.g. bankstatementconverter.com, docparser, LLM extraction APIs). Paid/cloud is not acceptable even as a default.
2. Must be usable from Python.
3. License MUST be permissive & commercially safe: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL or that depends on copyleft native components ? in particular Camelot/tabula's reliance on Ghostscript (AGPL) or a Java runtime. PyMuPDF (AGPL) is already banned for us.

Specifically evaluate and compare these table-extraction approaches, for each giving: license (AND license of native deps), maintenance status (latest release + activity as of 2025/2026), reliability on SEMI-RULED / partially-borderless tables (HDFC's transaction table is not fully line-ruled), and password-protected-PDF support:

- pdfplumber (we already use it; MIT)
- Camelot ? camelot-py vs the maintained "pypdf_table_extraction" fork; its Ghostscript/OpenCV deps
- tabula-py (tabula-java + JVM)
- pypdfium2-based table extraction
- any other notable local options

Also assess the user's idea: "convert the PDF table to Excel/CSV first, then parse" ? is an intermediate CSV/Excel extraction more reliable than direct coordinate/text parsing, and which LOCAL+permissive tools do PDF?CSV/Excel table conversion well?

Finally, search for any existing, maintained, permissively-licensed open-source projects specifically for parsing Indian bank statements (HDFC/ICICI/SBI/Axis) or bank statements generally, that we could reuse or learn from (GitHub/PyPI).

Deliver a ranked shortlist of the most promising + reliable + license-safe options for HDFC statement parsing in a local Python tool, each with its license (and native-dep licenses), maintenance status, strengths/weaknesses for this use case, and a concrete recommendation for muneem (reuse library X vs. hand-roll coordinate clustering with pdfplumber). Cite sources.

Claim under review

"Camelot supports password-protected/encrypted PDFs natively via a password argument to read_pdf(), with encryption handled by the playa library (no Java runtime), addressing the password-protected-PDF requirement for Indian bank statements."

****Source:**** <https://camelot-py.readthedocs.io/> (primary)

****Supporting quote:**** "tables = camelot.read_pdf('foo.pdf', password='userpass')"

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-05-30T19:17:05.992Z)

I'll verify this claim about Camelot's password-protected PDF support. Let me check the primary source and look for the playa library claim specifically.

3. assistant (2026-05-30T19:17:09.036Z)

Now let me investigate the claim. I'll fetch the primary Camelot docs and search for the playa library connection.

4. user (2026-05-30T19:17:11.298Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

5. user (2026-05-30T19:17:17.404Z)

Web search results for query: "camelot-py read_pdf password protected PDF encryption support"

Links: [{"title": "Cannot use on password protected files · Issue #162 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/issues/162"}, {"title": "Quickstart ? Camelot 1.0.9 documentation - Read the Docs", "url": "https://camelot-py.readthedocs.io/en/master/user/quickstart.html"}, {"title": "NotImplementedError: only algorithm code 1 and 2 are supported · Issue #325 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/issues/325"}, {"title": "[MRG + 1] Add basic support for encrypted PDF files by rbares · Pull Request #180 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/pull/180/files/d623aeed48a7d21a0af91151e841e9537871dceb"}, {"title": "Camelot Python Documentation Overview | PDF | Parameter (Computer Programming) | Software Development", "url": "https://www.scribd.com/document/689711867/camelot-py-readthedocs-io-en-master"}, {"title": "Camelot: PDF Table Extraction for Humans ? Camelot 2.0.0rc1 documentation", "url": "https://camelot-py.readthedocs.io/"}, {"title": "Stop Fighting with PDF: The 7 Python Libraries You Should Know | by Alice Yang | Medium", "url": "https://medium.com/@alice.yang_10652/stop-fighting-with-pdf-the-7-python-libraries-you-should-know-eaf8a19c30e8"}, {"title": "Camelot Documentation Release 1.0.9 Vinayak Mehta Aug 10, 2025", "url": "https://camelot-py.readthedocs.io/_/downloads/en/stable/pdf/"}, {"title": "How to Encrypt and Decrypt PDF Files in Python - The Python Code", "url": "https://thepythoncode.com/article/encrypt-pdf-files-in-"}]

```
python"]}]
```

Based on the search results, here's what I found about camelot-py's support for password-protected PDF encryption:

Password Support in camelot-py

Camelot's `read_pdf()` function accepts a `password` parameter that can be used to decrypt password-protected PDFs. You can use it like this:

```
```python
tables = camelot.read_pdf('foo.pdf', password='userpass')
```
```

Camelot also supports this via the command-line interface with the `--password` flag.

Encryption Algorithm Support

Camelot supports PDFs with all encryption types supported by pypdf, though this might require installing PyCryptodome.

However, there are limitations: An exception is thrown if the PDF cannot be read due to no password being provided, an incorrect password, or an unsupported encryption algorithm, though further encryption support may be added in future.

Workaround for Unsupported Encryption

If your PDF files are using unsupported encryption algorithms, you are advised to remove encryption before calling `read_pdf()`, which can be successfully achieved with third-party tools such as QPDF.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

6. assistant (2026-05-30T19:17:21.013Z)

The search confirms the `password` argument exists and that decryption was historically handled by `**pypdf**` (not `playa`). Let me verify the `playa` claim specifically ? this is the key dependency assertion in the claim. Let me check the current quickstart docs and the Camelot 2.0/master source.

7. user (2026-05-30T19:17:22.923Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of `WebFetch`.

8. user (2026-05-30T19:17:29.025Z)

Web search results for query: "camelot-py playa pdf encryption decryption password dependency"

Links: [{"title": "Cannot use on password protected files · Issue #162 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/issues/162"}, {"title": "Camelot Documentation Release 1.0.9 Vinayak Mehta Aug 10, 2025", "url": "https://camelot-py.readthedocs.io/_/downloads/en/stable/pdf/"}, {"title": "Quickstart ? Camelot 1.0.9 documentation - Read the Docs", "url": "https://camelot-py.readthedocs.io/en/master/user/quickstart.html"}, {"title": "Camelot Python Documentation Overview | PDF | Parameter (Computer Programming) | Software Development", "url": "https://www.scribd.com/document/689711867/camelot-py-readthedocs-io-en-master"}, {"title": "NotImplementedError: only algorithm code 1 and 2 are supported · Issue #325 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/issues/325"}, {"title": "[MRG + 1] Add

```
basic support for encrypted PDF files by rbares · Pull Request #180 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/pull/180/files/d623aeed48a7d21a0af91151e841e9537871dceb"}, {"title": "camelot.handlers ? Camelot 1.0.9 documentation", "url": "https://camelot-py.readthedocs.io/en/master/_modules/camelot/handlers.html"}, {"title": "Encryption and Decryption of PDFs ? pypdf 6.12.1 documentation", "url": "https://pypdf.readthedocs.io/en/stable/user/encryption-decryption.html"}, {"title": "Python Data Extraction from an Encrypted PDF_python_Mangs-Python", "url": "https://devpress.csdn.net/python/63045e2cc67703293080bed1.html"}, {"title": "camelot/handlers.py", "url": "https://www.diffchecker.com/KYgveBwc/"}]
```

Based on the search results, here's what I found about camelot-py's PDF encryption and decryption support:

Encryption/Decryption Support

Camelot supports PDFs with all encryption types supported by pypdf, which might require installing PyCryptodome.

Password Parameter

To extract tables from encrypted PDF files you must provide a password when calling `read_pdf()`. The password for decryption can be specified in the command-line interface.

Dependencies

The library relies on the `**pypdf**` library for encryption/decryption functionality. Camelot uses PdfReader and PdfWriter from the pypdf library to handle PDF operations, including checking if files are encrypted and decrypting them.

Error Handling

An exception is thrown if the PDF cannot be read, which may be due to no password being provided, an incorrect password, or an unsupported encryption algorithm. If your PDF files are using unsupported encryption algorithms, you are advised to remove encryption before calling `read_pdf()` using third-party tools such as QPDF.

Key Notes

- The underlying pypdf library determines which encryption algorithms are supported
- PyCryptodome may need to be installed separately for certain encryption types
- The password parameter is optional and defaults to an empty string if not provided

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

9. assistant (2026-05-30T19:17:32.707Z)

This is the critical finding. The documentation and search results consistently state that Camelot uses `**pypdf**` (PdfReader/PdfWriter) for encryption/decryption ? NOT playa. Let me verify directly from the source code handler to be certain about the decryption mechanism, since the claim specifically attributes it to playa.

10. user (2026-05-30T19:17:34.613Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. ``gh`` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

11. user (2026-05-30T19:17:41.408Z)

Web search results for query: "playa-pdf library python what is it PDF parser"

Links: [{"title": "playa-pdf · PyPI", "url": "https://pypi.org/project/playa-pdf/"}, {"title": "GitHub - dhdaines/playa: Parallel and LAZY Analyzer for PDFs", "url": "https://github.com/dhdaines/playa"}, {"title": "py-pdf-parser", "url": "https://pypi.org/project/py-pdf-parser/"}, {"title": "Overview - PDF Parser documentation - Read the Docs", "url": "https://py-pdf-parser.readthedocs.io/en/latest/overview.html"}, {"title": "pdfreader · PyPI", "url": "https://pypi.org/project/pdfreader/"}, {"title": "GitHub - py-pdf/pypdf: A pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files", "url": "https://github.com/py-pdf/pypdf"}, {"title": "Parse PDFs and other data formats in Python", "url": "https://konfuzio.com/en/pdf-parsing-python/"}, {"title": "GitHub - jstockwin/py-pdf-parser: A Python tool to help extracting information from structured PDFs.", "url": "https://github.com/jstockwin/py-pdf-parser"}, {"title": "pypdf | Python Library for Efficient PDF Parsing", "url": "https://products.documentprocessing.com/parser/python/pypdf/"}]

Based on the search results, here's what the **playa-pdf library** is:

Overview

Playa is a "Parallel and LAZY Analyzer for PDFs" ? a Python PDF parser library. PLAYA (using 2 CPUs) is the fastest pure-Python PDF reader by far and is also very good at reading logical structure trees.

Primary Use Cases

The library is designed for accessing the document catalog, page tree, structure tree, outline, content streams, cross-reference table, XObjects, fonts, images, annotations, and other low-level PDF metadata, as well as obtaining the absolute position and attributes of every character, line, path, and image in every page of a PDF.

Key Features

Parallel Processing: PLAYA allows you to take advantage of multiple CPUs, which can greatly speed up some operations on large documents, with parallelism currently operating at the page level.

Lazy Evaluation: Information can be computed lazily wherever possible.

What It's Not Best For

If you just want to extract text from a PDF, there are better and/or faster tools and libraries out there, notably pypdfium2 and pypdf, among others. PLAYA is the fastest pure-Python PDF reader overall, but text extraction isn't its primary focus.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

12. assistant (2026-05-30T19:17:44.626Z)

Now I have strong evidence. Let me verify the source code directly via GitHub to settle whether playa or pypdf handles decryption in the maintained Camelot 2.0/master codebase. The 404s on readthedocs `\_modules` mean I should go to the actual repo (`atlanhq/camelot` is the maintained one per search results).

13. user (2026-05-30T19:17:50.882Z)

Import Statements

```
```python
import os
```

```

import sys
from PyPDF2 import PdfFileReader, PdfFileWriter
from .core import TableList
from .parsers import Stream, Lattice
from .utils import (
 TemporaryDirectory,
 get_page_layout,
 get_text_objects,
 get_rotation,
 is_url,
 download_url,
)
...

```

## Password and Encrypted PDF Handling

The code uses **PyPDF2** for decryption. The relevant implementation appears in the `__init__` method and `_get_pages` method:

```

Password setup:
```python
if infile.isEncrypted:
    infile.decrypt(self.password)
...

```

The library imported is `PyPDF2`, specifically using its `PdfFileReader` class. The decryption workflow checks the `isEncrypted` property and calls the `decrypt()` method with the provided password to unlock encrypted documents before processing.

14. user (2026-05-30T19:17:52.508Z)

Issue Summary

This GitHub issue documents a problem where Camelot fails when processing certain PDF files with an error message: `***NotImplementedError: only algorithm code 1 and 2 are supported***`

The Core Problem

The error occurs not because the PDF is password-protected (the user explicitly notes theirs isn't), but because the PDF uses an **encryption algorithm that Camelot's underlying PyPDF2 library doesn't support**.

Specifically, the PyPDF2 library only handles PDF encryption algorithms numbered 1 and 2. When a PDF uses algorithm version 3 or higher, the decryption process fails, even if no password is involved.

Key Point

The issue title is somewhat misleading?this isn't fundamentally about password protection. Rather, it's about PDF encryption standard compatibility. The Camelot library attempts to decrypt any encrypted PDF during processing, and it crashes when encountering modern encryption schemes beyond what PyPDF2 can handle.

15. assistant (2026-05-30T19:17:56.925Z)

I now have conclusive evidence. Let me note the version nuance: the `master` `handlers.py` shows PyPDF2 (the older atlanhq lineage), but search results reference "Camelot 2.0.0rc1" and "1.0.9" docs citing pypdf. Either way, the decryption library is PyPDF2/pypdf ? never playa. Let me do one final confirmation check on whether the newer/renamed fork uses pypdf and whether playa appears anywhere in Camelot's encryption path.

16. user (2026-05-30T19:18:01.623Z)

I cannot find the words 'playa', 'pypdf', 'PyPDF2', or 'PyCryptodome' in the provided document content. The document appears to be a PDF file with compressed/encoded stream data rather than readable text about Python libraries or PDF handling.

To answer your question accurately, I would need a document containing actual readable text discussing these libraries and their encryption-handling capabilities.

[Binary content (application/pdf, 1.2MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\9fd4426-f234-4e4a-bfb4-1f2c8255c661\tool-results\webfetch-1780168673195-ajgfbk.pdf]

17. user (2026-05-30T19:18:05.143Z)

Web search results for query: ""camelot" pypdf encrypted PDF "PyCryptodome" password read_pdf 2025"

Links: [{"title": "Encryption and Decryption of PDFs - pypdf - Read the Docs", "url": "https://pypdf.readthedocs.io/en/stable/user/encryption-decryption.html"}, {"title": "Python Data Extraction from an Encrypted PDF_python_Mangs-Python", "url": "https://devpress.csdn.net/python/63045e2cc67703293080bed1.html"}, {"title": "Cannot use on password protected files · Issue #162 · atlanhq/camelot", "url": "https://github.com/atlanhq/camelot/issues/162"}, {"title": "Camelot Documentation Release 1.0.9 Vinayak Mehta Aug 10, 2025", "url": "https://camelot-py.readthedocs.io/_/downloads/en/stable/pdf/"}, {"title": "Encryption and Decryption of PDFs ? pypdf 3.12.0 documentation", "url": "https://pypdf.readthedocs.io/en/3.12.0/user/encryption-decryption.html"}, {"title": "Hemant Vishwakarma: Python Data Extraction from an Encrypted PDF", "url": "https://hemant9807.blogspot.com/2019/10/python-data-extraction-from-encrypted.html"}, {"title": "Encrypt and Decrypt PDFs with pypdf in Python | note.nkmk.me", "url": "https://note.nkmk.me/en/python-pypdf2-pdf-password/"}, {"title": "Quickstart ? Camelot 1.0.9 documentation - Read the Docs", "url": "https://camelot-py.readthedocs.io/en/master/user/quickstart.html"}]

I've found search results related to your query about using Camelot, pypdf, encrypted PDFs, and PyCryptodome. Here are the key findings:

Encryption Support in Camelot

Camelot supports PDFs with all encryption types supported by pypdf, which might require installing PyCryptodome. An exception is thrown if the PDF cannot be read, which may be due to no password being provided, an incorrect password, or an unsupported encryption algorithm.

pypdf and PyCryptodome

pypdf uses an extra dependency to do encryption or decryption for AES algorithms, and you can use pycryptodome as an alternative. The more modern and secure AES encryption requires pycryptodome to be installed.

Password-Protected PDFs

For password-protected PDFs, you must already know the password to decrypt a file; pypdf doesn't support password cracking or bypassing. If your PDF files are using unsupported encryption algorithms you are advised to remove encryption before calling read_pdf(), which can be successfully achieved with third-party tools such as QPDF.

Important Note

The Camelot repository was archived by the owner on Jan 6, 2025 and is now read-only. This means the project is no longer actively maintained.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. assistant (2026-05-30T19:18:17.432Z)

I have conclusive, consistent evidence across multiple primary sources. Let me assess the claim against the checklist.

****The claim has two parts:****

1. "Camelot supports password-protected PDFs natively via a `password` argument to `read\_pdf()`" ? ****TRUE** and supported by the quote. **** Multiple sources confirm `camelot.read\_pdf('foo.pdf', password='userpass')` works.**
2. "with encryption handled by the ****playa**** library (no Java runtime)" ? ****FALSE.**** This is the fabricated/unsupported part.

****Evidence the "playa" attribution is wrong:****

- The Camelot source (`atlanhq/camelot/master/camelot/handlers.py`) imports `from PyPDF2 import PdfFileReader, PdfFileWriter` and decrypts via `infile.decrypt(self.password)`. No playa.
- The current docs (Camelot 1.0.9 / 2.0.0rc1) state: "Camelot supports PDFs with all encryption types supported by ****pypdf****, which might require installing ****PyCryptodome****." Not playa.
- Issue #325's "NotImplementedError: only algorithm code 1 and 2 are supported" comes from PyPDF2's limited encryption support ? a pypdf-lineage limitation, not playa.
- `playa-pdf` (dhdaines/playa) is a separate, unrelated parser ("Parallel and LAzY Analyzer"); the search confirms its purpose is fast parsing/layout, and nothing connects it to Camelot's decryption path.

****The supporting quote**** (`camelot.read\_pdf('foo.pdf', password='userpass')`) proves only the `password` argument ? it says ****nothing**** about playa. So the "playa handles encryption" mechanism is unsupported by the cited quote and contradicted by the actual code/docs.

****Additional refutation context:**** The native dep is actually pypdf/PyPDF2 (BSD/permissive) plus optional PyCryptodome (BSD) ? and modern AES-encrypted PDFs (which many Indian bank statements use) hit the "only algorithm code 1 and 2 are supported" failure in older lineages, so even the practical reliability is qualified. Also the `atlanhq/camelot` repo was archived Jan 6, 2025.

The claim is ****REFUTED****: the mechanism it asserts (playa handling encryption) is fabricated. Decryption is handled by pypdf/PyPDF2, not playa. The "no Java runtime" part happens to be true (Camelot is pure-Python for PDF I/O), but it's attached to a false causal claim about playa.

19. user (2026-05-30T19:18:27.708Z)

Structured output provided successfully

20. assistant (2026-05-30T19:18:40.849Z)

REFUTED (high confidence).

The claim is a partial truth wrapped around a fabricated mechanism. Breaking it down:

****Supported part:**** Camelot does accept a `password` argument to `read\_pdf()` ?
`camelot.read\_pdf('foo.pdf', password='userpass')` is real. The cited quote proves only this.

****Refuted part ? "encryption handled by the playa library":**** This is false and unsupported.

- Camelot's actual source (`atlanhq/camelot/master/camelot/handlers.py`) imports `from PyPDF2 import PdfFileReader, PdfFileWriter` and decrypts via `infile.decrypt(self.password)`. No playa anywhere.
- Current Camelot docs (1.0.9 / 2.0.0rc1) state: "Camelot supports PDFs with all encryption types supported by ****pypdf****, which might require installing ****PyCryptodome****." The decryption dependency is pypdf/PyPDF2, not playa.
- `playa-pdf` (dhdaines/playa, "Parallel and LAzY Analyzer for PDFs") is a separate, unrelated pure-Python parser focused on layout/structure analysis ? nothing ties it to Camelot's decryption path.
- The cited supporting quote mentions only the `password` arg; it says nothing about playa, so the mechanism is an overreach/fabrication, not a misread of the quote.

****Two failure modes from the checklist hit:**** (1) the strong mechanistic claim is unsupported by the quote, and (2) it is contradicted by the primary source (the code itself).

****Caveats relevant to muneem regardless of the verdict:****

- The "no Java runtime" sub-claim is incidentally true (Camelot's PDF I/O is pure-Python via pypdf/PyPDF2), but it rides on the false playa attribution.
- Practical reliability is qualified: Issue #325 ("NotImplementedError: only algorithm code 1 and 2 are supported") shows the pypdf/PyPDF2 lineage chokes on modern AES-encrypted PDFs ? exactly what many Indian bank statements use. Workaround is to strip encryption with QPDF first.
- License note for muneem: pypdf (BSD) and PyCryptodome (BSD/public-domain) are permissive ? but Camelot's **table-extraction** reliability still depends on Ghostscript (AGPL, for Lattice) / OpenCV, which is the separate license concern flagged in the research question. And ``atlanhq/camelot`` was archived Jan 6, 2025.

Sources:

- <https://raw.githubusercontent.com/atlanhq/camelot/master/camelot/handlers.py>
- <https://camelot-py.readthedocs.io/en/master/user/quickstart.html>
- <https://github.com/atlanhq/camelot/issues/325>
- <https://github.com/dhdaines/playa>